ASSIGNMENT-12.5

Task Description #1 (Sorting – Merge Sort Implementation):

Prompt:

```
#generate a python program to o generate a Python program that implements the Merge Sort algorithm A functional Python script implementing Merge Sort with proper documentation.
```

Code:

```python
def merge_sort(arr):
    """
    Sorts an array using the Merge Sort algorithm.

    Parameters:
    arr (list): The list of elements to be sorted.

    Returns:
    list: A new sorted list.
    """
    if len(arr) <= 1:
        return arr

    # Split the array into halves
    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    # Merge the sorted halves
    return merge(left_half, right_half)
def merge(left, right):
    """
    Merges two sorted lists into a single sorted list.

    Parameters:
    left (list): The first sorted list.
    right (list): The second sorted list.

    Returns:
    list: A merged and sorted list.
    """
    merged = []
    left_index = right_index = 0

    # Merge the two lists while maintaining sorted order
    while left_index < len(left) and right_index < len(right):
        if left[left_index] < right[right_index]:
```

```
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    # If there are remaining elements in the left half, add them
    while left_index < len(left):
        merged.append(left[left_index])
        left_index += 1

    # If there are remaining elements in the right half, add them
    while right_index < len(right):
        merged.append(right[right_index])
        right_index += 1

    return merged
# Example usage
if __name__ == "__main__":
    unsorted_array = [38, 27, 43, 3, 9, 82, 10]
    sorted_array = merge_sort(unsorted_array)
    print("Sorted array:", sorted_array)
```

output:

```
Sorted array: [3, 9, 10, 27, 38, 43, 82]
```

Justification:

The program implements the **Merge Sort algorithm** to sort a list in ascending order. It divides the list into smaller parts, sorts them recursively, and merges them back in order. This approach ensures correct sorting. The algorithm runs in **O(n log n)** time and the test case confirms the output.

Task Description #2 (Searching – Binary Search with AI

Optimization):

Prompt:

```
#generate a python program to create a binary search function
that finds a target element in a sorted list Python code
implementing binary search with AI- generated comments and
docstrings.
```

Code:

```python
def binary_search(arr, target):
    """
    Performs binary search on a sorted list to find the target element.

    Parameters:
    arr (list): A sorted list of elements.
    target: The element to search for.

    Returns:
    int: The index of the target element if found, otherwise -1.
    """
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2

        # Check if the target is present at mid
        if arr[mid] == target:
            return mid
        # If target is greater, ignore the left half
        elif arr[mid] < target:
            left = mid + 1
        # If target is smaller, ignore the right half
        else:
            right = mid - 1

    # Target was not found in the array
    return -1
# Example usage
if __name__ == "__main__":
    sorted_array = [1, 2, 3, 4, 5, 6, 7, 8, 9]
    target_element = 5
    result = binary_search(sorted_array, target_element)

    if result != -1:
        print(f"Element {target_element} found at index: {result}")
    else:
        print(f"Element {target_element} not found in the array.")
```

Output:

```
Element 5 found at index: 4
```

Justification:

Binary Search works only on a sorted list and finds the target element by repeatedly dividing the list into two halves.

This method is faster and efficient compared to linear search for large datasets.

Task Description #3: Smart Healthcare Appointment Scheduling System:

Prompt:

```
#Recommend suitable searching and sorting algorithms for a
Smart Healthcare Appointment Scheduling System to search by
appointment ID and sort by time or consultation fee, justify
the choice, and implement it in Python.
```

Code:

```python
def binary_search(arr, target):
    """
    Performs binary search on a sorted list to find the target element.

    Parameters:
    arr (list): A sorted list of elements.
    target: The element to search for.

    Returns:
    int: The index of the target element if found, otherwise -1.
    """
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2

        # Check if the target is present at mid
        if arr[mid] == target:
            return mid
        # If target is greater, ignore the left half
        elif arr[mid] < target:
            left = mid + 1
        # If target is smaller, ignore the right half
        else:
            right = mid - 1

    # Target was not found in the array
    return -1
# Implementing Merge Sort for sorting by time or consultation fee
def merge_sort(arr, key):
    """
    Sorts an array of dictionaries based on a specified key using the Merge Sort
algorithm.

    Parameters:
```

```
    arr (list): The list of dictionaries to be sorted.
    key (str): The key in the dictionary to sort by.

    Returns:
    list: A new sorted list of dictionaries.
    """
    if len(arr) <= 1:
        return arr

    # Split the array into halves
    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid], key)
    right_half = merge_sort(arr[mid:], key)

    # Merge the sorted halves
    return merge(left_half, right_half, key)
def merge(left, right, key):
    """
    Merges two sorted lists of dictionaries into a single sorted list based on a
specified key.

    Parameters:
    left (list): The first sorted list of dictionaries.
    right (list): The second sorted list of dictionaries.
    key (str): The key in the dictionary to sort by.

    Returns:
    list: A merged and sorted list of dictionaries.
    """
    merged = []
    left_index = right_index = 0

    # Merge the two lists while maintaining sorted order
    while left_index < len(left) and right_index < len(right):
        if left[left_index][key] < right[right_index][key]:
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    # If there are remaining elements in the left half, add them
    while left_index < len(left):
        merged.append(left[left_index])
        left_index += 1

    # If there are remaining elements in the right half, add them
    while right_index < len(right):
        merged.append(right[right_index])
        right_index += 1
```

```python
    return merged
# Example usage
if __name__ == "__main__":
    appointments = [
        {"id": 1, "time": "10:00", "fee": 100},
        {"id": 2, "time": "11:00", "fee": 150},
        {"id": 3, "time": "12:00", "fee": 80},
    ]
    # Sort appointments by time
    sorted_by_time = merge_sort(appointments, key="time")
    print("Appointments sorted by time:", sorted_by_time)

    # Sort appointments by fee
    sorted_by_fee = merge_sort(appointments, key="fee")
    print("Appointments sorted by fee:", sorted_by_fee)

    # Search for an appointment by ID
    appointment_ids = [appointment["id"] for appointment in appointments]
    target_id = 2
    index = binary_search(appointment_ids, target_id)
    if index != -1:
        print(f"Appointment with ID {target_id} found at index: {index}")
    else:
        print(f"Appointment with ID {target_id} not found.")
```

Output:

```
Appointments sorted by time: [{'id': 1, 'time': '10:00', 'fee': 100}, {'id': 2, 'time': '11:00', 'fee': 150}, {'id': 3, 'time': '12:00', 'fee': 80}]
Appointments sorted by fee: [{'id': 3, 'time': '12:00', 'fee': 80}, {'id': 1, 'time': '10:00', 'fee': 100}, {'id': 2, 'time': '11:00', 'fee': 150}]
Appointment with ID 2 found at index: 1
```

Justification:

Binary Search helps to quickly find an appointment using the appointment ID in a sorted list. Merge Sort efficiently sorts the appointments based on time or consultation fee for better scheduling and management.

Task Description #4: Railway Ticket Reservation System

Scenario:

Prompt:

```
# write efficient searching and sorting algorithms for a
Railway Ticket Reservation System and implement Python
functions to search tickets by ticket ID and sort bookings by
travel date or seat number, including justification of the
chosen algorithms.
```

Code:

```python
def binary_search(arr, target, key):
    """
    Performs binary search on a sorted list of dictionaries to find the target
element based on a specified key.

    Parameters:
    arr (list): A sorted list of dictionaries.
    target: The value to search for.
    key (str): The key in the dictionary to compare against the target.

    Returns:
    int: The index of the target element if found, otherwise -1.
    """
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2

        # Check if the target is present at mid
        if arr[mid][key] == target:
            return mid
        # If target is greater, ignore the left half
        elif arr[mid][key] < target:
            left = mid + 1
        # If target is smaller, ignore the right half
        else:
            right = mid - 1

    # Target was not found in the array
    return -1
# Implementing Merge Sort for sorting by travel date or seat number
def merge_sort(arr, key):
    """
    Sorts an array of dictionaries based on a specified key using the Merge Sort
algorithm.

    Parameters:
    arr (list): The list of dictionaries to be sorted.
    key (str): The key in the dictionary to sort by.

    Returns:
```

```
    list: A new sorted list of dictionaries.
    """
    if len(arr) <= 1:
        return arr

    # Split the array into halves
    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid], key)
    right_half = merge_sort(arr[mid:], key)

    # Merge the sorted halves
    return merge(left_half, right_half, key)
def merge(left, right, key):
    """
    Merges two sorted lists of dictionaries into a single sorted list based on a
specified key.

    Parameters:
    left (list): The first sorted list of dictionaries.
    right (list): The second sorted list of dictionaries.
    key (str): The key in the dictionary to sort by.

    Returns:
    list: A merged and sorted list of dictionaries.
    """
    merged = []
    left_index = right_index = 0

    # Merge the two lists while maintaining sorted order
    while left_index < len(left) and right_index < len(right):
        if left[left_index][key] < right[right_index][key]:
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    # If there are remaining elements in the left half, add them
    while left_index < len(left):
        merged.append(left[left_index])
        left_index += 1

    # If there are remaining elements in the right half, add them
    while right_index < len(right):
        merged.append(right[right_index])
        right_index += 1

    return merged
# Example usage
if __name__ == "__main__":
    bookings = [
```

```python
        {"ticket_id": "A123", "travel_date": "2024-07-01", "seat_number": 5},
        {"ticket_id": "B456", "travel_date": "2024-06-15", "seat_number": 3},
        {"ticket_id": "C789", "travel_date": "2024-07-10", "seat_number": 1}
    ]

    # Sort bookings by travel date
    sorted_by_date = merge_sort(bookings, key="travel_date")
    print("Bookings sorted by travel date:")
    for booking in sorted_by_date:
        print(booking)

    # Search for a ticket by ticket ID
    target_ticket_id = "B456"
    index = binary_search(sorted_by_date, target_ticket_id, key="ticket_id")
    if index != -1:
        print(f"Ticket with ID {target_ticket_id} found: {sorted_by_date[index]}")
    else:
        print(f"Ticket with ID {target_ticket_id} not found.")
    # Sort bookings by seat number
    sorted_by_seat = merge_sort(bookings, key="seat_number")
    print("\nBookings sorted by seat number:")
    for booking in sorted_by_seat:
        print(booking)
    # Search for a ticket by seat number
    target_seat_number = 3
    index = binary_search(sorted_by_seat, target_seat_number, key="seat_number")
    if index != -1:
        print(f"Ticket with seat number {target_seat_number} found:
{sorted_by_seat[index]}")
    else:
        print(f"Ticket with seat number {target_seat_number} not found.")
```

Output:

```
Bookings sorted by travel date:
{'ticket_id': 'B456', 'travel_date': '2024-06-15', 'seat_number': 3}
{'ticket_id': 'A123', 'travel_date': '2024-07-01', 'seat_number': 5}
{'ticket_id': 'C789', 'travel_date': '2024-07-10', 'seat_number': 1}
Ticket with ID B456 not found.

Bookings sorted by seat number:
{'ticket_id': 'C789', 'travel_date': '2024-07-10', 'seat_number': 1}
{'ticket_id': 'B456', 'travel_date': '2024-06-15', 'seat_number': 3}
{'ticket_id': 'A123', 'travel_date': '2024-07-01', 'seat_number': 5}
Ticket with seat number 3 found: {'ticket_id': 'B456', 'travel_date': '2024-06-15', 'seat_number': 3}
```

Justification:

Binary Search efficiently finds a ticket using the ticket ID or seat number in a sorted list. Merge Sort organizes

bookings by travel date or seat number, making reservation management easier and faster.

Task Description #5: Smart Hostel Room Allocation System:

Prompt:

```
#Recommend optimized searching and sorting algorithms for a
Smart Hostel Room Allocation System and implement Python
functions to search allocation details by student ID and sort
records by room number or allocation date, including
justification of the chosen algorithms.
```

Code:

```
class RoomAllocation:
    def __init__(self, student_id, room_number, allocation_date):
        self.student_id = student_id
        self.room_number = room_number
        self.allocation_date = allocation_date
class HostelRoomAllocationSystem:
    def __init__(self):
        self.allocations = []
    def add_allocation(self, allocation):
        self.allocations.append(allocation)
    def search_allocation_by_student_id(self, student_id):
        self.allocations.sort(key=lambda x: x.student_id)  # Ensure allocations are
sorted by Student ID
        left, right = 0, len(self.allocations) - 1
        while left <= right:
            mid = (left + right) // 2
            if self.allocations[mid].student_id == student_id:
                return self.allocations[mid]
            elif self.allocations[mid].student_id < student_id:
                left = mid + 1
            else:
                right = mid - 1
        return None
    def sort_allocations_by_room_number(self):
        self.allocations.sort(key=lambda x: x.room_number)
    def sort_allocations_by_allocation_date(self):
        self.allocations.sort(key=lambda x: x.allocation_date)
# Example usage
hostel_system = HostelRoomAllocationSystem()
hostel_system.add_allocation(RoomAllocation("S001", "101", "2023-09-01"))
```

```
hostel_system.add_allocation(RoomAllocation("S002", "102", "2023-09-02"))
hostel_system.add_allocation(RoomAllocation("S003", "101", "2023-09-03"))
print(hostel_system.search_allocation_by_student_id("S002").room_number)  # Output:
102
hostel_system.sort_allocations_by_room_number()
print([allocation.student_id for allocation in hostel_system.allocations])  #
Output: ['S001', 'S003', 'S002']
hostel_system.sort_allocations_by_allocation_date()
print([allocation.student_id for allocation in hostel_system.allocations])  #
Output: ['S001', 'S002', 'S003']
```

Output:

```
102
['S001', 'S003', 'S002']
['S001', 'S002', 'S003']
```

Justification:

Binary Search is used to efficiently find a student allocation by student ID in (O(\log n)) time after sorting the records.Python's built-in Timsort is used for sorting by room number or allocation date because it is stable, efficient, and has (O(n \log n)) time complexity.

Task Description #6: Online Movie Streaming Platform:

Prompt:

```
#Recommend suitable searching and sorting algorithms for an
Online Movie Streaming Platform and implement Python functions
to search movies by movie ID and sort movies by rating or
release year, with justification of the chosen algorithms.
```

Code:

```
class Movie:
    def __init__(self, movie_id, title, rating, release_year):
        self.movie_id = movie_id
```

```
        self.title = title
        self.rating = rating
        self.release_year = release_year
class MovieStreamingPlatform:
    def __init__(self):
        self.movies = []
    def add_movie(self, movie):
        self.movies.append(movie)
    def search_movie_by_id(self, movie_id):
        self.movies.sort(key=lambda x: x.movie_id)  # Ensure movies are sorted by
Movie ID
        left, right = 0, len(self.movies) - 1
        while left <= right:
            mid = (left + right) // 2
            if self.movies[mid].movie_id == movie_id:
                return self.movies[mid]
            elif self.movies[mid].movie_id < movie_id:
                left = mid + 1
            else:
                right = mid - 1
        return None
    def sort_movies_by_rating(self):
        self.movies.sort(key=lambda x: x.rating, reverse=True)  # Sort by rating in
descending order
    def sort_movies_by_release_year(self):
        self.movies.sort(key=lambda x: x.release_year)  # Sort by release year in
ascending order
# Example usage
platform = MovieStreamingPlatform()
platform.add_movie(Movie("M001", "Inception", 8.8, 2010))
platform.add_movie(Movie("M002", "The Matrix", 8.7, 1999))
platform.add_movie(Movie("M003", "Interstellar", 8.6, 2014))
print(platform.search_movie_by_id("M002").title)  # Output: The Matrix
platform.sort_movies_by_rating()
print([movie.title for movie in platform.movies])  # Output: ['Inception', 'The
Matrix', 'Interstellar']
platform.sort_movies_by_release_year()
print([movie.title for movie in platform.movies])  # Output: ['The Matrix',
'Inception', 'Interstellar']
```

Output:

```
The Matrix
['Inception', 'The Matrix', 'Interstellar']
['The Matrix', 'Inception', 'Interstellar']
```

Justification:

Binary Search helps to quickly find a movie using the movie ID in a sorted list.Sorting movies by rating or release year helps users easily view top-rated and latest movies.

Task Description #7: Smart Agriculture Crop Monitoring System:

Prompt:

```
#Recommend suitable searching and sorting algorithms for a
Smart Agriculture Crop Monitoring System and implement in
Python to search crops by crop ID and sort crops by soil
moisture level or yield estimate, including justification of
the chosen algorithms.
```

Code:

```
class Crop:
    def __init__(self, crop_id, soil_moisture, yield_estimate):
        self.crop_id = crop_id
        self.soil_moisture = soil_moisture
        self.yield_estimate = yield_estimate
class CropMonitoringSystem:
    def __init__(self):
        self.crops = []
    def add_crop(self, crop):
        self.crops.append(crop)
    def search_crop_by_id(self, crop_id):
        self.crops.sort(key=lambda x: x.crop_id)  # Ensure crops are sorted by Crop
ID
        left, right = 0, len(self.crops) - 1
        while left <= right:
            mid = (left + right) // 2
            if self.crops[mid].crop_id == crop_id:
                return self.crops[mid]
            elif self.crops[mid].crop_id < crop_id:
                left = mid + 1
            else:
                right = mid - 1
        return None
    def sort_crops_by_soil_moisture(self):
        self.crops.sort(key=lambda x: x.soil_moisture)
```

```python
    def sort_crops_by_yield_estimate(self):
        self.crops.sort(key=lambda x: x.yield_estimate)
# Example usage
cms = CropMonitoringSystem()
cms.add_crop(Crop("C001", 30, 100))
cms.add_crop(Crop("C002", 20, 150))
cms.add_crop(Crop("C003", 25, 120))
print(cms.search_crop_by_id("C002").yield_estimate)  # Output: 150
cms.sort_crops_by_soil_moisture()
print([crop.crop_id for crop in cms.crops])  # Output: ['C002', 'C003', 'C001']
cms.sort_crops_by_yield_estimate()
print([crop.crop_id for crop in cms.crops])  # Output: ['C001', 'C003', 'C002']
```

Output:

```
150
['C002', 'C003', 'C001']
['C001', 'C003', 'C002']
```

Justification:

Binary Search helps to quickly find a crop using the crop ID in a sorted list. Sorting crops by soil moisture level helps farmers monitor irrigation needs. Sorting by yield estimate helps in analysing and planning better crop production.

Task Description #8: Airport Flight Management System:

Prompt:

```
#Recommend suitable searching and sorting algorithms for an
Airport Flight Management System and implement in Python to
search flights by Flight ID and sort flights by departure and
arrival time with justification
```

Code:

```python
class Flight:
    def __init__(self, flight_id, departure_time, arrival_time):
        self.flight_id = flight_id
        self.departure_time = departure_time
        self.arrival_time = arrival_time
class FlightManagementSystem:
    def __init__(self):
        self.flights = []
    def add_flight(self, flight):
        self.flights.append(flight)
    def search_flight_by_id(self, flight_id):
        self.flights.sort(key=lambda x: x.flight_id)  # Ensure flights are sorted
by Flight ID
        left, right = 0, len(self.flights) - 1
        while left <= right:
            mid = (left + right) // 2
            if self.flights[mid].flight_id == flight_id:
                return self.flights[mid]
            elif self.flights[mid].flight_id < flight_id:
                left = mid + 1
            else:
                right = mid - 1
        return None
    def sort_flights_by_departure_time(self):
        self.flights.sort(key=lambda x: x.departure_time)
    def sort_flights_by_arrival_time(self):
        self.flights.sort(key=lambda x: x.arrival_time)
# Example usage
fms = FlightManagementSystem()
fms.add_flight(Flight("AA123", "08:00", "10:00"))
fms.add_flight(Flight("BB456", "09:00", "11:00"))
fms.add_flight(Flight("CC789", "07:00", "09:00"))
print(fms.search_flight_by_id("BB456").departure_time)  # Output: 09:00
fms.sort_flights_by_departure_time()
print([flight.flight_id for flight in fms.flights])  # Output: ['CC789', 'AA123',
'BB456']
```

Output:

```
09:00
['CC789', 'AA123', 'BB456']
```

Justification:

Binary Search is used to efficiently find a flight by Flight ID in (O(\log n)) time after sorting the records. Python's built-in timesort is used for sorting by departure time or arrival

time because it is stable and efficient with $(O(n \log n))$ time complexity.